# 23.03.26 tuple,set,union

In [1]:
```
l3=[30,10,40,2]
l3
```

Out[1]:
```
[30, 10, 40, 2]
```

In [2]:
```
l3.sort()
```

In [3]:
```
l3
```

Out[3]:
```
[2, 10, 30, 40]
```

In [9]:
```
l3.sort(reverse=True)
l3
```

Out[9]:
```
[40, 30, 10, 2]
```

In [10]:
```
l4=[2,1+5j,True, 3.5]
l4
```

Out[10]:
```
[2, (1+5j), True, 3.5]
```

In [6]:
```
l4.sort()  # it means sorting method in list is applicable for same datatype onl
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[6], line 1
----> 1 l4.sort()  # it means sorting method in list is applicable for same datat
ype only.

TypeError: '<' not supported between instances of 'complex' and 'int'
```

In [11]:
```
l5=['z','m','b','c','x'] # here in character you have to apply apostraphe comma
l5.sort()
l5
```

Out[11]:
```
['b', 'c', 'm', 'x', 'z']
```

In [12]:
```
l5.sort(reverse=True)
l5
```

Out[12]:
```
['z', 'x', 'm', 'c', 'b']
```

In [ ]:
```
# list indexing  and slicing
```

In [13]:
```
l1=[10,2.3,(1+2j), True,'nit',[1,2,3]]
l1
```

Out[13]:
```
[10, 2.3, (1+2j), True, 'nit', [1, 2, 3]]
```

In [14]:
```
l1[:]
```

Out[14]: [10, 2.3, (1+2j), True, 'nit', [1, 2, 3]]

In [15]:
```
l1[::-2]
```

Out[15]: [[1, 2, 3], True, 2.3]

In [16]:
```
l1[2:]
```

Out[16]: [(1+2j), True, 'nit', [1, 2, 3]]

In [17]:
```
l1
```

Out[17]: [10, 2.3, (1+2j), True, 'nit', [1, 2, 3]]

In [ ]:
```
l1[0:10:3]
```

In [18]:
```
l1
```

Out[18]: [10, 2.3, (1+2j), True, 'nit', [1, 2, 3]]

In [19]:
```
l1[2]=35  # list is mutable
l1
```

Out[19]: [10, 2.3, 35, True, 'nit', [1, 2, 3]]

# tuple

In [20]:
```
t=()
t
```

Out[20]: ()

In [21]:
```
type(t)
```

Out[21]: tuple

In [22]:
```
t1=(10,20,30)
t1
```

Out[22]: (10, 20, 30)

In [23]:
```
t1.count(10)
```

Out[23]: 1

In [24]:
```
t1.count(100)
```

Out[24]: 0

In [25]:
```
t1.index(0)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[25], line 1
----> 1 t1.index(0)

ValueError: tuple.index(x): x not in tuple
```

In [26]:
```
t1.index(10)
```

Out[26]:
```
0
```

In [27]:
```
t1[0]=23  # insertion is not allowed in tuple
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[27], line 1
----> 1 t1[0]=23  # insertion is not allowed in tuple

TypeError: 'tuple' object does not support item assignment
```

In [28]:
```
t1
```

Out[28]:
```
(10, 20, 30)
```

In [29]:
```
t1*3 # just repeat  no of times not multiply.
```

Out[29]:
```
(10, 20, 30, 10, 20, 30, 10, 20, 30)
```

# set

In [30]:
```
s={}
s
```

Out[30]:
```
{}
```

In [ ]:
```
type(s)  # {} creating an empty dictionary not set.
```

In [31]:
```
s1=set()
s1
```

Out[31]:
```
set()
```

In [32]:
```
s1={10,30,20,50,40} # set by default sort in ascending order
s1
```

Out[32]:
```
{10, 20, 30, 40, 50}
```

In [33]:
```
s1.add(100)
s1
```

Out[33]:
```
{10, 20, 30, 40, 50, 100}
```

In [35]:
```
s1.add(15)
```

In [36]:
```
s1
```

Out[36]: {10, 15, 20, 30, 40, 50, 100}

In [37]:
```
s2 = {5, 5.5, 'nit', 1+2j, True}
s2
```

Out[37]: {(1+2j), 5, 5.5, True, 'nit'}

In [38]:
```
s2.add(50)
s2
```

Out[38]: {(1+2j), 5, 5.5, 50, True, 'nit'}

In [39]:
```
s2.add(100) # added ascending order only when there is similar datatype otherwis
s2
```

Out[39]: {(1+2j), 100, 5, 5.5, 50, True, 'nit'}

In [40]:
```
print(s1)
print(s2)
```

```
{50, 20, 100, 40, 10, 30, 15}
{True, 50, 100, 5, 'nit', 5.5, (1+2j)}
```

In [41]:
```
s3=s2.copy()
s3
```

Out[41]: {(1+2j), 100, 5, 5.5, 50, True, 'nit'}

In [42]:
```
s3[0]  # indexing works only in list, tuple, string but not in set because order
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[42], line 1
----> 1 s3[0]  # indexing works only in list, tuple, string but not in set becaus
e order is not fixed in set.

TypeError: 'set' object is not subscriptable
```

In [ ]:
```
s3
```

In [43]:
```
s3[:] # set=unordered, slicing only happens in orderered data types
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[43], line 1
----> 1 s3[:] # set=unordered, slicing only happens in orderered data types

TypeError: 'set' object is not subscriptable
```

In [44]:
```
s3
```

Out[44]: {(1+2j), 100, 5, 5.5, 50, True, 'nit'}

In [45]:
```
s3.pop() # removes random one element from the set, sets are unordered so you ca
```

Out[45]: True

In [46]:
```
s3.remove(5.5)
s3
```

Out[46]:
```
{(1+2j), 100, 5, 50, 'nit'}
```

In [47]:
```
s3.remove(200)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[47], line 1
----> 1 s3.remove(200)

KeyError: 200
```

In [48]:
```
s3
```

Out[48]:
```
{(1+2j), 100, 5, 50, 'nit'}
```

In [49]:
```
s3.discard()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[49], line 1
----> 1 s3.discard()

TypeError: set.discard() takes exactly one argument (0 given)
```

In [56]:
```
s3.remove()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[56], line 1
----> 1 s3.remove()

TypeError: set.remove() takes exactly one argument (0 given)
```

In [50]:
```
s3.discard('nit')
s3
```

Out[50]:
```
{(1+2j), 100, 5, 50}
```

In [51]:
```
s3.discard(200)  # if element not present in list discard dont give error.
s3
```

Out[51]:
```
{(1+2j), 100, 5, 50}
```

In [ ]:
```
s2
```

In [52]:
```
s2.add(100) # in set duplicate is not allowed
```

In [53]:
```
s2
```

Out[53]:
```
{(1+2j), 100, 5, 5.5, 50, True, 'nit'}
```

In [54]:
```
s2.add(200)
s2
```

Out[54]: {(1+2j), 100, 200, 5, 5.5, 50, True, 'nit'}

In [55]:
```
s2.remove(1+2j)
s2
```

Out[55]: {100, 200, 5, 5.5, 50, True, 'nit'}

# set operation

In [59]:
```
s7 = {1,2,3,4,5}
s8 = {4,5,6,7,8}
s9 = {8,9,10}
```

In [60]:
```
s7.union(s8)
```

Out[60]: {1, 2, 3, 4, 5, 6, 7, 8}

In [61]:
```
s8.union(s9)
```

Out[61]: {4, 5, 6, 7, 8, 9, 10}

In [62]:
```
s7.union(s8,s9)
```

Out[62]: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

In [63]:
```
s8|s9
```

Out[63]: {4, 5, 6, 7, 8, 9, 10}

In [64]:
```
s8&s9
```

Out[64]: {8}

In [66]:
```
s7|s8|s9
```

Out[66]: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

In [67]:
```
s7&s8&s9
```

Out[67]: set()

In [68]:
```
s7&s8
```

Out[68]: {4, 5}

In [69]:
```
s7
s8
```

Out[69]: {4, 5, 6, 7, 8}

In [70]:
```
print(s7)
print(s8)
print(s9)
```

```
{1, 2, 3, 4, 5}
{4, 5, 6, 7, 8}
{8, 9, 10}
```

In [73]:
```
s7.intersection(s8)
```

Out[73]:
```
{4, 5}
```

In [74]:
```
s8.intersection(s9)
```

Out[74]:
```
{8}
```

In [ ]: